🎭 ORANGEHRM AUTOMATION FRAMEWORK

Enterprise-Grade End-to-End QA Automation

Playwright **TypeScript** **JMeter** **Allure Reports** **GitHub Actions**

Presented by: **Abhay Singh** | QA Automation Engineer

Framework covers: E2E · API · Visual · Accessibility · Performance · Smoke

📋 AGENDA

01 — BUSINESS PROBLEM
Why manual testing is not enough at scale

02 — FRAMEWORK OVERVIEW
Architecture, goals, and key decisions

03 — TECH STACK
Tools chosen and why

04 — TEST TYPES
7 testing dimensions covered

05 — PAGE OBJECT MODEL
How the framework is structured

06 — CI/CD PIPELINE
Automated execution on GitHub Actions

07 — REPORTING
Allure, HTML, PDF, Slack, Email

08 — PERFORMANCE TESTING
JMeter suite — load profiles and metrics

09 — LIVE DEMO COMMANDS
How to run locally and in CI

🚨 THE BUSINESS PROBLEM

❌ MANUAL TESTING CHALLENGES

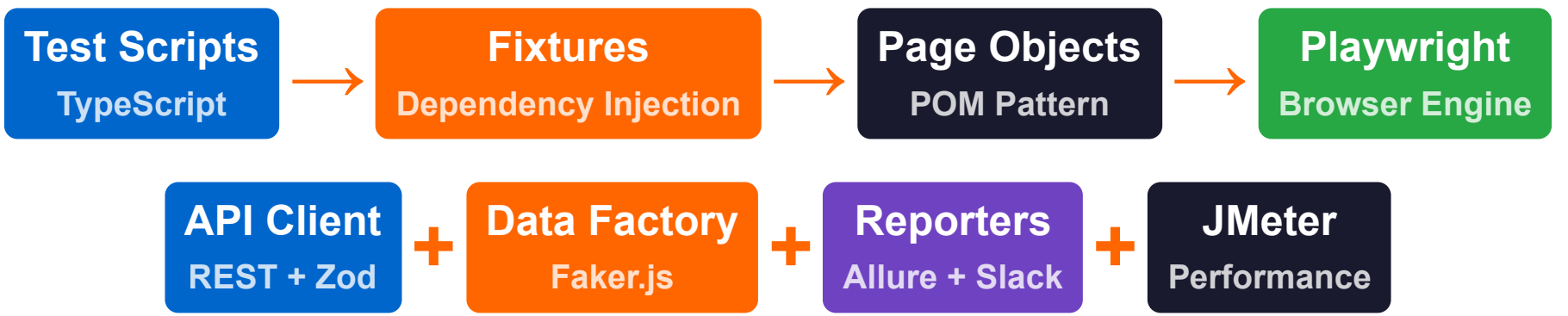
- Regression cycles take 3–5 days per release
- Human error rate increases with repetition
- No cross-browser consistency
- No API-level validation
- Zero performance visibility
- No audit trail for accessibility compliance

✅ WHAT AUTOMATION SOLVES

- Full regression in < 15 minutes
- Zero human error on repeated flows
- Chrome + Firefox + Safari + Mobile in parallel
- REST API validated at schema level
- Response time benchmarks every night
- WCAG 2.1 compliance verified automatically

Goal: Ship every release with confidence — every commit is validated across browsers, APIs, visual layout, accessibility, and performance.

🏗️ FRAMEWORK ARCHITECTURE



🔒 TYPESCRIPT STRICT MODE

Full type safety across the entire codebase — runtime errors caught at compile time.

📦 PAGE OBJECT MODEL

UI interaction logic is isolated from test logic — tests read like plain English.

🔧 FIXTURE INJECTION

Custom Playwright fixtures provide pre-built pages, API clients, and test data to every test automatically.

🛠 TECH STACK

Technology	Version	Purpose	Why Chosen
Playwright	^1.57.0	Cross-browser automation	Native async, auto-waits, network interception, multi-browser
TypeScript	^5.6.0	Language	Strict typing catches errors before runtime; IDE autocomplete
Node.js	20.x LTS	Runtime	Long-term support, fast npm, CI-standard
Allure	^2.15.0	Test reporting	Rich HTML reports with steps, screenshots, history trends
axe-core	^4.11.3	Accessibility	Industry-standard WCAG checker — zero-config integration
Faker.js	^8.0.2	Test data	Realistic, randomised data — no hardcoded test values
Zod	^4.4.3	Schema validation	Type-safe API response validation at runtime

Technology	Version	Purpose	Why Chosen
Apache JMeter	5.6.3	Performance testing	Industry-standard load tool, CI-friendly, free, generates HTML/PDF reports
GitHub Actions	Latest	CI/CD	Native GitHub integration, matrix builds, artifact storage

🧪 7 TESTING DIMENSIONS

🔐
SMOKE
Login critical path — fast gate

🌐
E2E
Full user journeys across all modules

⚡
API
REST endpoints + schema validation

👁️
VISUAL
Pixel-level baseline comparison

♿
A11Y
WCAG 2.1 accessibility compliance

🌐
NETWORK
Mock, intercept, simulate failures

📊
PERF
JMeter load + response time

Smoke runs in <2 min as a CI gate — blocks the pipeline if login is broken.

API tests validate both happy path and error responses at the HTTP level — independent of UI.

Visual tests detect unintended UI regressions — CSS, layout, and component changes.

PAGE OBJECT MODEL (POM)

STRUCTURE

```
// src/pages/LoginPage.ts export class
LoginPage extends BasePage { async
login(user: string, pass: string) {
await
this.page.fill('[name=username]',
user); await
this.page.fill('[name=password]',
pass); await
this.page.click('[type=submit]');
await
this.page.waitForURL('/dashboard'); }
} // tests/auth/login.spec.ts
test('login with valid credentials',
async ({ loginPage }) => { await
loginPage.login('Admin', 'admin123');
await
```

KEY PRINCIPLES

1. **One class per page** — LoginPage, DashboardPage, EmployeePage each own their selectors and actions.
2. **BasePage** provides shared utilities: navigation, waits, toast reading, screenshot helpers.
3. **Tests are selector-free** — no locators in test files; tests describe behaviour, pages handle mechanics.
4. **Single point of change** — if a selector changes, update one method in one class, not 50 test files.

```
expect(page).toHaveURL(/dashboard/);
});
```

Result: Tests read like acceptance criteria — easy for non-engineers to review and validate.

🔧 FIXTURES & TEST DATA

CUSTOM FIXTURES

Fixtures provide pre-built objects to every test via dependency injection — no boilerplate setup in tests.

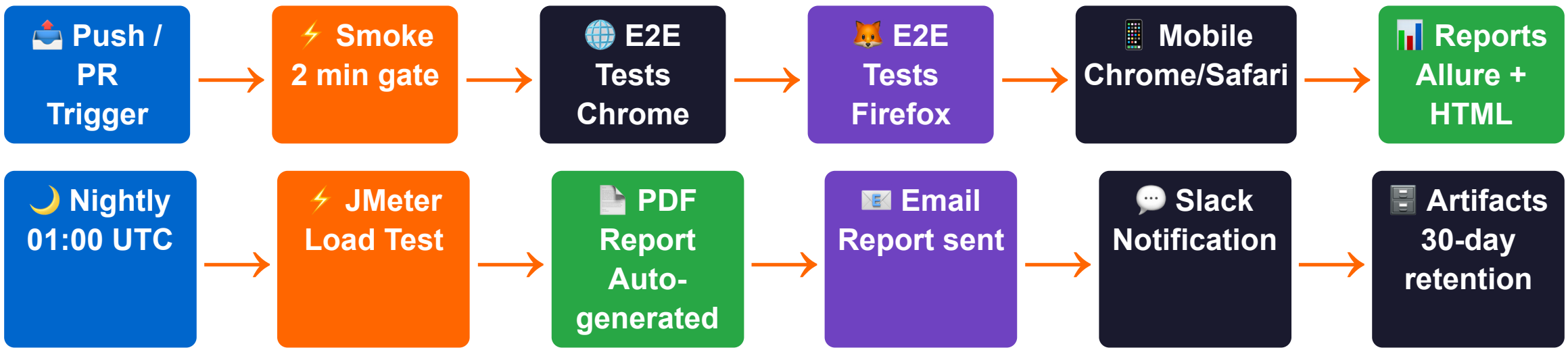
Fixture	Provides
`loginPage`	LoginPage instance
`dashboardPage`	DashboardPage instance
`employeePage`	EmployeePage instance
`apiClient`	Typed REST client
`employeeData`	Faker-generated employee
`authenticatedPage`	Pre-logged-in browser

DATA FACTORY

```
// src/helpers/DataFactory.ts export function createEmployee() { return { firstName: faker.person.firstName(), lastName: faker.person.lastName(), employeeId: faker.string.numeric(6), email: faker.internet.email(), }; }
```

No hardcoded test data. Every test run uses unique, realistic data generated by Faker.js — eliminates test pollution and data conflicts.

🔄 CI/CD PIPELINE — GITHUB ACTIONS



🔀 BROWSER MATRIX

Chrome, Firefox, WebKit, Mobile Chrome, Mobile Safari — all run in parallel on every PR.

🔄 RETRY LOGIC

Flaky tests are automatically retried up to 2 times in CI — reduces false alarm failures.

📦 ARTIFACT STORAGE

Test results, Allure reports, and performance PDFs uploaded and retained for 30 days per run.

📊 REPORTING SUITE

🏆 ALLURE REPORT

Rich test reports with:

- Pass/fail breakdown per suite
- Step-by-step execution trace
- Screenshots on failure
- Historical trend graphs
- Severity and epic tagging

🌐 PLAYWRIGHT HTML

Built-in Playwright report:

- Timeline view
- Video replay on failure
- Network request log
- Trace viewer (step-by-step)
- Shareable static HTML

📄 JMETER PDF + HTML

Performance reports:

- Response time charts
- Throughput graphs
- P90/P99 percentiles
- Error rate timeline
- One-click PDF for clients

📊 JUNIT XML

Machine-readable format:

- Compatible with any CI dashboard
- Jenkins, Azure DevOps, TeamCity
- Pass/fail counts per test class

💬 SLACK NOTIFICATIONS

Real-time alerts:

- Pass ✅ / Fail ❌ per run
- Test counts + failure summary
- Direct link to report
- Configurable webhook

📧 EMAIL REPORTS

Automated email delivery:

- PDF performance report attached
- Run summary in email body
- Triggered by CI on

⚡ PERFORMANCE TESTING HIGHLIGHTS

JMETER LOGIN SUITE

1. **GET** /auth/login — Page load time
2. **POST** /auth/validateCredentials — Auth server response
3. **GET** /dashboard/index — Post-login dashboard
4. **GET** /auth/logout — Session cleanup

3 load profiles:

🔵 Smoke: 1 user, 1 loop

🟠 Baseline: 5 users, 10 loops, 10s ramp

🔴 Stress: 50 users, 20 loops, 60s ramp

TARGET THRESHOLDS

Metric	Target
Avg Response Time	< 2,000 ms
P90 Response Time	< 3,000 ms
P99 Response Time	< 5,000 ms
Error Rate	0% (max 1%)
Latency (TTFB)	< 500 ms
Throughput	Maximised

Nightly CI run keeps a **continuous performance baseline** — regressions detected before they reach production.

🚀 HOW TO RUN

SETUP

```
# 1. Clone and install git clone
github.com/qaabhayraj-
cpu/DemoPlaywriteAI-AutomationProject
npm ci npx playwright install # 2.
Configure environment cp
config/environments/qa.env .env
```

PLAYWRIGHT TESTS

```
# Smoke gate (fast — 2 min) npm run
test:smoke # Full regression npm run
test:regression # By browser npm run
test:chrome npm run test:firefox npm
run test:webkit # API tests only npm
run test:api # Visual + A11y npm run
test:visual npm run test:a11y
```

PERFORMANCE TESTS

```
# One-time JMeter setup npm run
perf:setup # Run login load test npm
run perf:login # Custom: 10 users, 5
loops bash jmeter/scripts/run.sh login
\ -Jthreads=10 -Jloops=5 -Jramp_up=30
# Open HTML dashboard npm run
perf:report:login # Generate PDF npm
run perf:pdf
```

REPORTS

```
# Playwright HTML report npm run
report # Allure report npm run
report:allure npm run report:open
```

📁 PROJECT STRUCTURE

```
DemoPlaywriteAI-AutomationProject/ ├── src/ │ ├── pages/ ← Page Object Model │ ├── fixtures/ ← Dependency injection │ ├── api/ ← REST API client │ ├── helpers/ ← Utilities & data │ └── types/ ← TypeScript interfaces ├── tests/ │ ├── smoke/ ← Fast CI gate │ ├── auth/ ← Login tests │ ├── employee/ ← CRUD tests │ ├── api/ ← API tests │ ├── visual/ ← Screenshot diff │ ├── a11y/ ← WCAG compliance │ └── network/ ← Request mocking ├── jmeter/ │ ├── test-plans/ ← login.jmx │ └── scripts/ ← setup, run, pdf, email ├── config/ │ └── environments/ ← dev / qa / staging ├── .github/workflows/ ← CI/CD pipelines └── playwright.config.ts
```

📁 SRC/ — FRAMEWORK CORE

All reusable logic: page objects, API clients, helpers, type definitions. Tests never contain selectors or raw fetch calls.

📁 TESTS/ — TEST SUITES

Pure test logic — describe what should happen, not how. 7 test types, each in its own folder.

📁 JMETER/ — PERFORMANCE

Self-contained load testing suite: plans, scripts, and auto-generated results/reports folders.

📁 CONFIG/ — ENVIRONMENTS

dev.env / qa.env / staging.env — switch environments with a single env var:

```
TEST_ENV=staging npm test
```

✅ VALUE DELIVERED

7
TEST TYPES
E2E, API, Visual, A11y, Network, Smoke, Performance

5
BROWSERS
Chrome, Firefox, Safari, Mobile Chrome, Mobile Safari

<15m
FULL REGRESSION
Complete test suite in under 15 minutes on CI

3
ENVIRONMENTS
Dev, QA, Staging — zero config changes to switch

24/7
MONITORING
Nightly CI runs — performance baseline every morning

PDF
CLIENT REPORTS
Auto-generated performance PDF ready for stakeholders

0
MANUAL SETUP
npm ci + npm test — runs anywhere, first try

WCAG
ACCESSIBILITY
axe-core validates WCAG 2.1 compliance automatically



QUESTIONS & DISCUSSION

Playwright TypeScript JMeter GitHub Actions

📧 qaabhayraj@gmail.com

🔗 github.com/qaabhayraj-cpu/DemoPlaywriteAI-AutomationProject

Quick start: `git clone … && npm ci && npm test`